数据结构

图

考试大纲

五、图

（一）图的基本概念

（二）图的存储及基本操作

1. 邻接矩阵

2.邻接表

3.邻接多重表、十字链表

（三）图的遍历

1.深度优先搜索

2.广度优先搜索

（四）图的基本应用

1.最小(代价)生成树

2.最短路径

3.拓扑排序

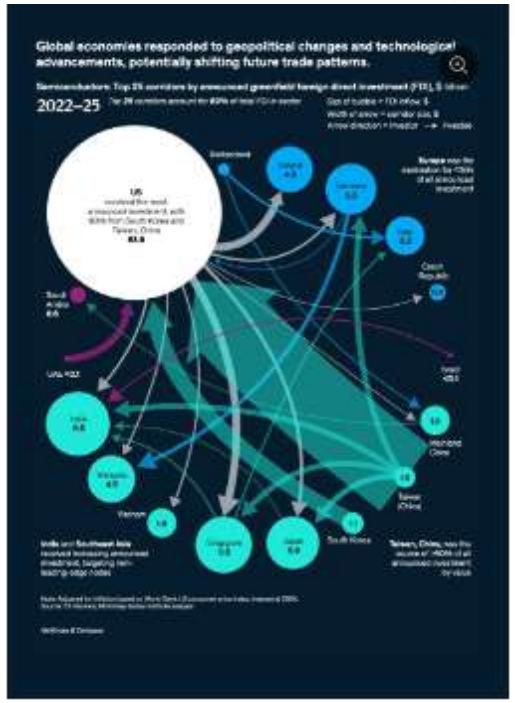
4.关键路径

图的逻辑定义 G = (V, E)

图由**顶点集 V** 和 **边集 E** 组成。

- **V(G)**：非空有限集合，表示顶点。
- **E(G)**：顶点之间的关系集合。

> 🚨 **考研陷阱：**
>
> 在图论中，顶点集 V 必须是**非空**的。但边集 E 可以为空。



有向图 vs 无向图

无向图 (Undirected)

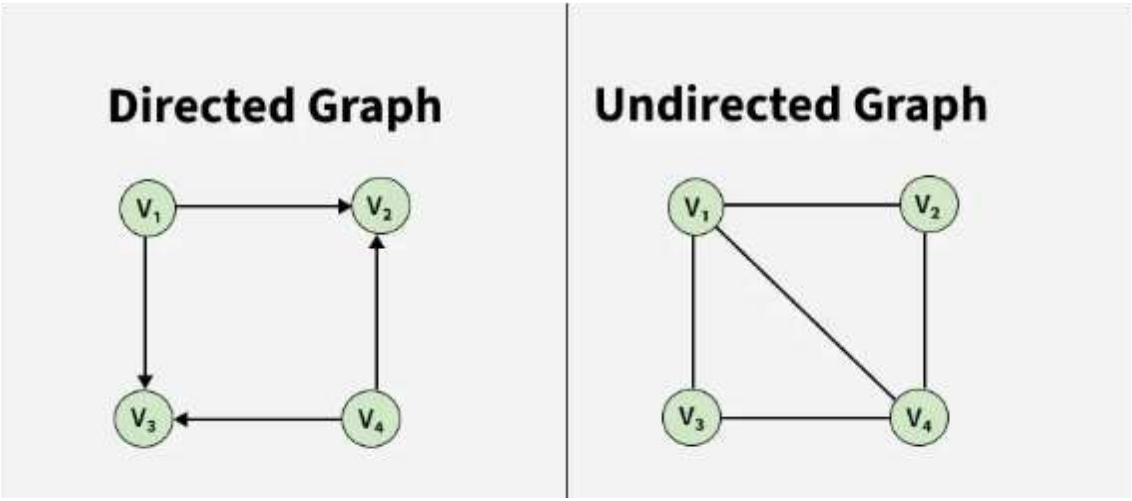
边是无方向的。表示为 (v, w)。

性质：(v, w) = (w, v)。

有向图 (Directed)

边是有方向的，称为“弧”。表示为 $\langle v, w \rangle$。

性质：v 是弧尾，w 是弧头。$\langle v, w \rangle \neq \langle w, v \rangle$。



简单图与完全图

简单图 (Simple Graph)

- 不存在重复边。
- 不存在顶点到自身的边（无环）。
- **图默认为简单图。**

完全图 (Complete)

任意两点间都存在边。

- 无向完全图边数：$n(n-1)/2$
- 有向完全图弧数：$n(n-1)$

子图与生成子图

设 G=(V, E)，G'=(V', E')。

- **子图**：若 $V' \subseteq V$ 且 $E' \subseteq E$。
- **生成子图**：满足 V' = V 的子图（即包含原图所有顶点）。

🚨 并非 V, E 的任意子集都能构成子图，必须保证边集 E' 中的每一条边的两个端点都在 V' 中。

顶点的度、入度与出度

无向图

顶点 v 的度为关联该顶点的边数。

定理： 度数之和 = 边数的 2 倍。

有向图

入度 (ID)：以 v 为头的弧数。

出度 (OD)：以 v 为尾的弧数。

总度 TD(v) = ID + OD。

连通图与连通分量 (无向)

连通图

图中任意两个顶点都是连通的。

极小连通图 = **生成树**。

连通分量

无向图中的**极大连通子图**。

注意：这里的“极大”是指尽可能包含更多的边和点。

强连通图与强连通分量 (有向)

对于有向图，若任意 v_i, v_j之间既有从 v_i 到 v_j 的路径，也有从 v_j 到 v_i 的路径，则称该图为**强连通图**。

> 🚨 **常见计算题考点：**
>
> 1. 一个有 n 个顶点的有向图，若是强连通图，最少有 n 条边（形成环）。
>
> 2. 一个有 n 个顶点的无向图，若是非连通图，最多可以有 (n-1)(n-2)/2 条边。

邻接矩阵：逻辑与实现

利用二维数组存储顶点之间的邻接关系。

- A[i][j] = 1：顶点 i 到 j 有边。
- A[i][j] = 0：无边。

特点：无向图矩阵是对称的。

Comparison of Adjacency list and Adjacency matrix

enjoyalgorithms.com

	Adjacency List	Adjacency Matrix
Time complexity to check if edge (u, v) present in the graph	O(d), here d is the adjacency list length of vertex u.	O(1)
Time complexity to find degree of a vertex in the graph	O(d)	O(V)
Space complexity to store graph	O(V + E)	O(V^2)
Time complexity to insert or delete an edge	O(d)	O(1)
Time complexity to traverse the graph	O(V + E)	O(V^2)

邻接矩阵：C 语言实现

```c
typedef struct {
  char Vex[MAXV];          // 顶点表
  int Edge[MAXV][MAXV];    // 邻接矩阵
  int vexnum, arcnum;      // 顶点数和边数
} MGraph;
```

空间复杂度：$O(|V|^2)$。适合**稠密图**。

邻接表：物理结构

为每个顶点建立一个单链表，存储该顶点的邻接点信息。

- **顶点表**：顺序存储（数组）。
- **边表**：链式存储。

空间复杂度：无向图 O(|V|+2|E|)；有向图 O(|V|+|E|)。

适合存储**稀疏图**。

缺陷：在有向图中找“入度”需要遍历全表，极不方便。

邻接表：核心数据结构

```
typedef struct ArcNode {    // 边表节点
  int adjvex; struct ArcNode *next;
} ArcNode;
typedef struct VNode {      // 顶点表节点
  char data; ArcNode *firstarc;
} VNode, AdjList[MAXV];
typedef struct {
  AdjList vertices; int vexnum, arcnum;
} ALGraph;
```

十字链表 (有向图优化)

解决了邻接表找入度困难的问题。每个弧对应一个节点，包含四个主要指针/域：

[tailvex | headvex | hlink | tlink]

- hlink：指向弧头相同的下一条弧。
- tlink：指向弧尾相同的下一条弧。



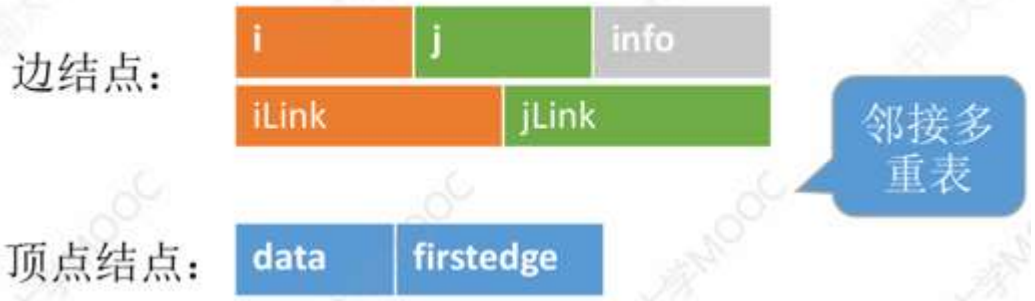
找入度和出度均可直接遍历对应链表。空间复杂度仍为 O(|V|+|E|)。

邻接多重表 (无向图优化)

解决了无向图邻接表中每一条边被存储两次（冗余且不便于标记删除）的问题。

每条边用一个节点表示：[i | ilink | j | jlink]

这样，所有的操作均针对同一个边节点进行。



图存储结构总结对比

结构	适用	复杂度	核心特点
邻接矩阵	稠密图	$O(V^2)$	查询边 O(1)，空间开销大
邻接表	稀疏图	O(V+E)	找邻接点快，找入度慢
十字链表	有向图	O(V+E)	结合了邻接表与逆邻接表
邻接多重表	无向图	O(V+E)	边不重复，便于删除边操作

图的基本操作

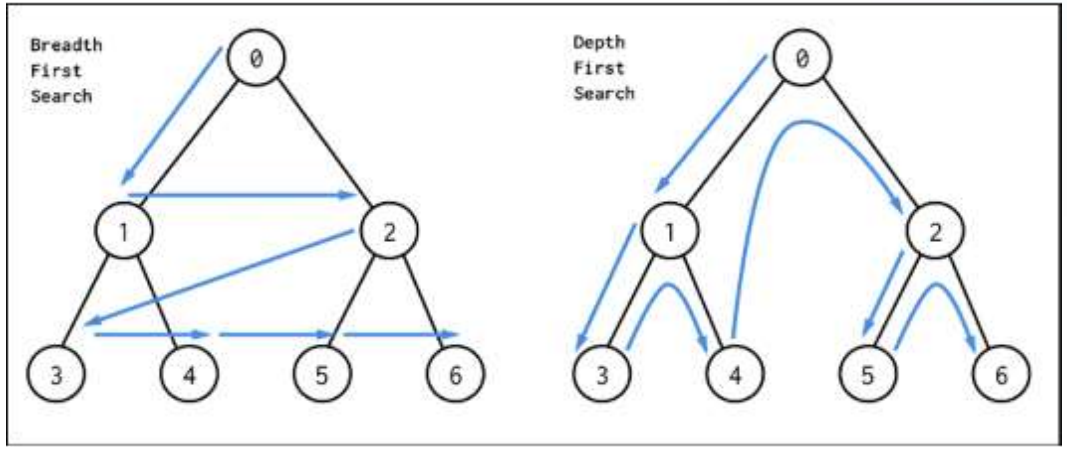
要求掌握以下操作的逻辑：

- Adjacent(G, x, y)：判断是否存在边。
- Neighbors(G, x)：列出所有邻接边。
- InsertVertex(G, x) & DeleteVertex(G, x)。
- Get_edge_value(G, x, y)。
- FirstNeighbor(G, x) / NextNeighbor(G, x, y) (遍历核心)。

广度优先搜索 (BFS) 逻辑

类似于树的**层序遍历**。借助**队列**实现。

1. 访问起始点 v，标记并入队。
2. 队不空，出队一个顶点 u。
3. 访问所有 u 未被标记的邻接点并入队。
4. 重复直到队空。




```
void BFS(Graph G, int v) {
  visit(v); visited[v] = true;
  Enqueue(Q, v);
  while(!isEmpty(Q)) {
    Dequeue(Q, u);
    for(w = FirstNeighbor(G, u); w >= 0; w = NextNeighbor(G, u, w)) {
      if(!visited[w]) {
        visit(w); visited[w] = true; Enqueue(Q, w);
      }
    }
  }
}
```

BFS 算法复杂度分析

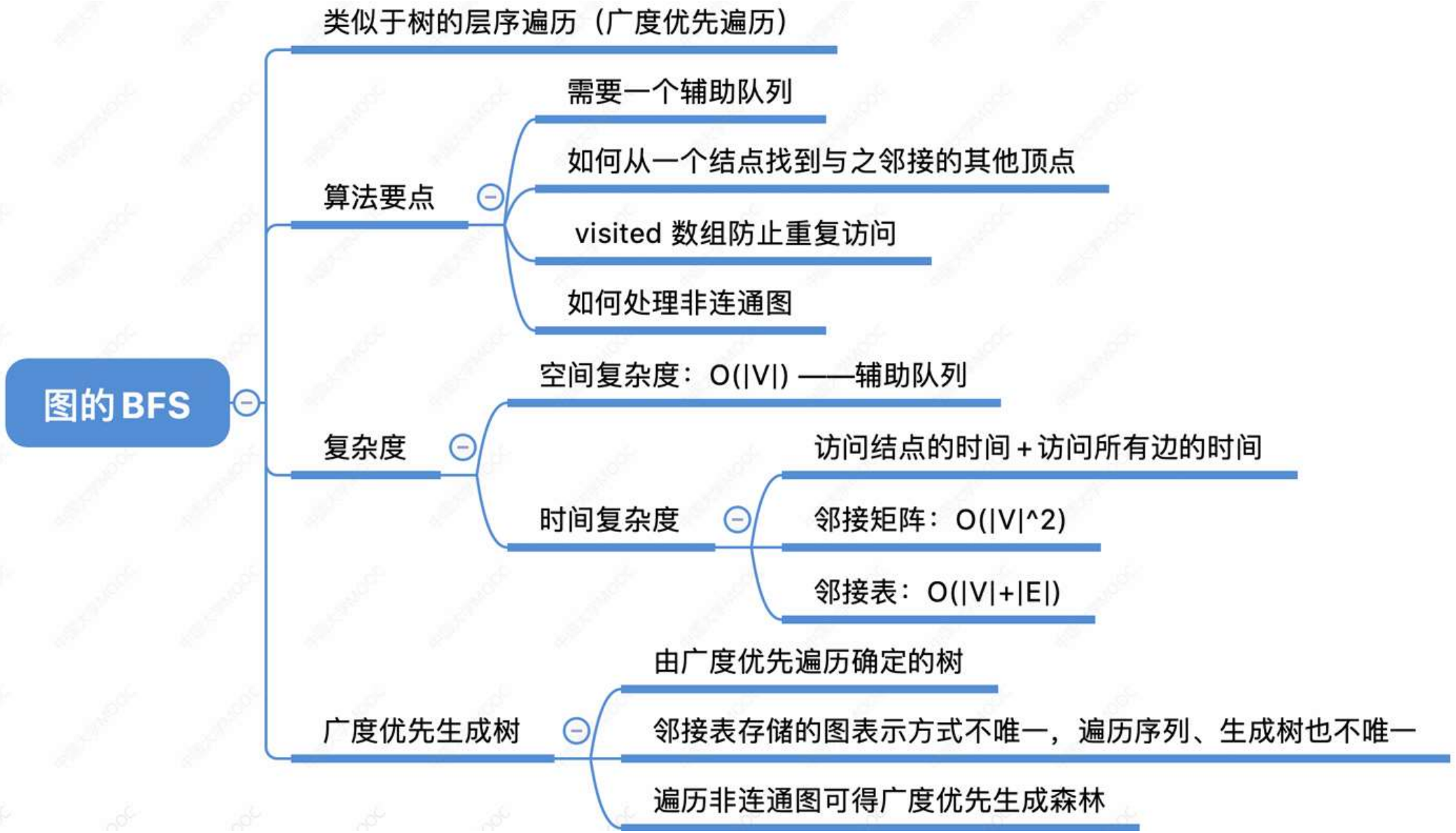
时间复杂度

- 邻接矩阵：$O(|V|^2)$
- 邻接表：O(|V|+|E|)

空间复杂度

最坏情况下 O(|V|)，取决于辅助队列的大小。

🚨 **性质：** BFS 可以用来求解非带权图的单源最短路径。


图的BFS
类似于树的层序遍历（广度优先遍历）
算法要点
需要一个辅助队列
如何从一个结点找到与之邻接的其他顶点
visited 数组防止重复访问
如何处理非连通图
复杂度
空间复杂度：O(|V|) ——辅助队列
时间复杂度
访问结点的时间 + 访问所有边的时间
邻接矩阵：O(|V|^2)
邻接表：O(|V|+|E|)
广度优先生成树
由广度优先遍历确定的树
邻接表存储的图表示方式不唯一，遍历序列、生成树也不唯一
遍历非连通图可得广度优先生成森林

深度优先搜索 (DFS) 逻辑

类似于树的**先序遍历**。借助**递归（栈）**实现。

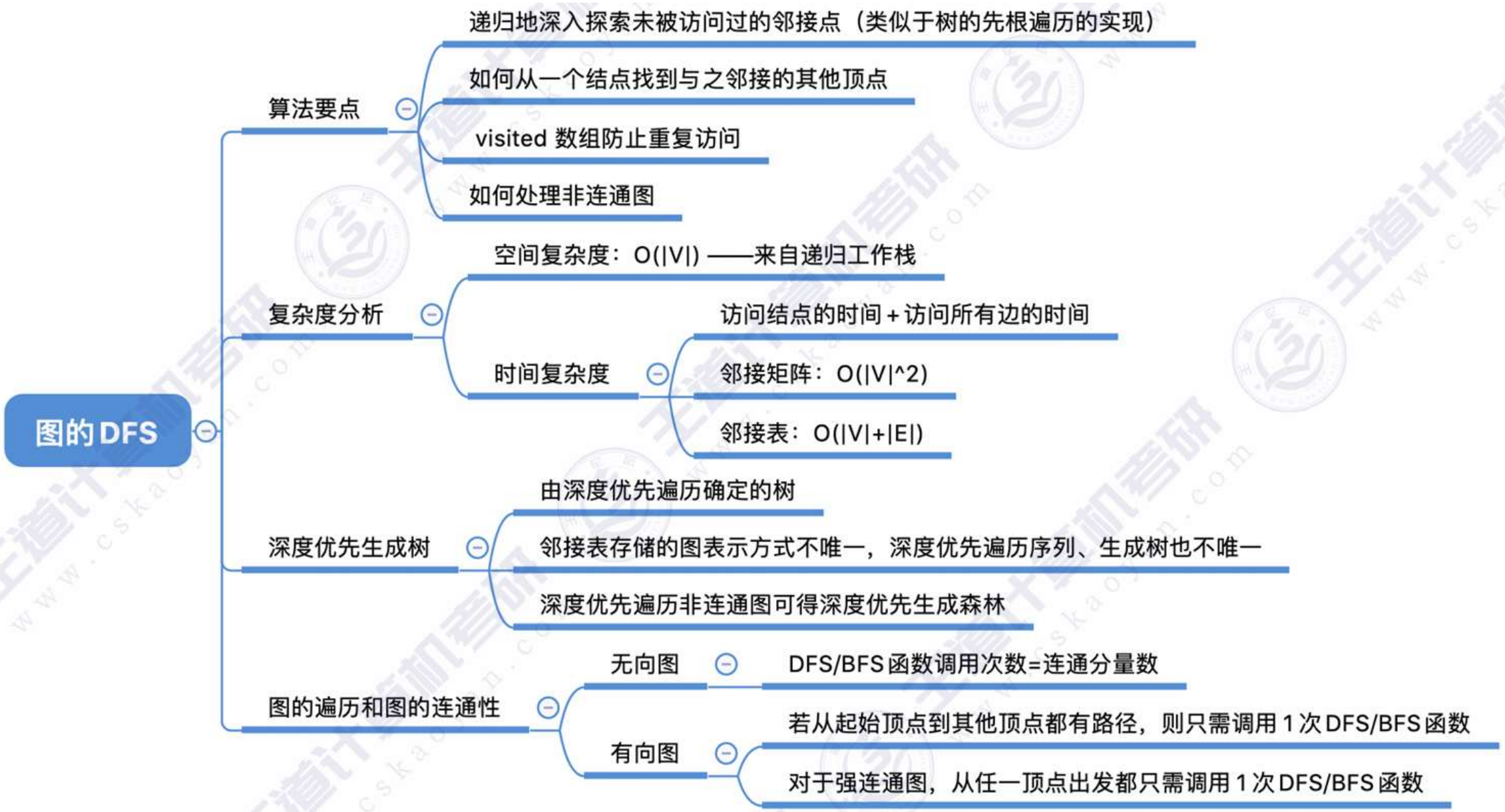
1. 访问顶点 v 并标记。
2. 依次从未被访问的邻接点出发继续深度搜索。
3. 若无法继续，则回溯到上一个顶点。

```
void DFS(Graph G, int v) {
  visit(v); visited[v] = true;
  for(w = FirstNeighbor(G, v); w >= 0; w = NextNeighbor(G, v, w))
    if(!visited[w]) DFS(G, w);
}
```

DFS 算法复杂度与序列性

时间复杂度与 BFS 相同，取决于存储结构。空间复杂度最坏为 O(|V|)。

> 🚨 **要点：**
>
> - 基于邻接表的遍历序列**不唯一**。
> - 基于邻接矩阵的遍历序列**唯一**。
> - 遍历过程中经历的边和点构成“生成树”或“生成森林”。


图的DFS
算法要点
递归地深入探索未被访问过的邻接点（类似于树的先根遍历的实现）
如何从一个结点找到与之邻接的其他顶点
visited 数组防止重复访问
如何处理非连通图
复杂度分析
空间复杂度：O(|V|) ——来自递归工作栈
时间复杂度
访问结点的时间 + 访问所有边的时间
邻接矩阵：O(|V|^2)
邻接表：O(|V|+|E|)
深度优先生成树
由深度优先遍历确定的树
邻接表存储的图表示方式不唯一，深度优先遍历序列、生成树也不唯一
深度优先遍历非连通图可得深度优先生成森林
图的遍历和图的连通性
无向图
DFS/BFS 函数调用次数=连通分量数
有向图
若从起始顶点到其他顶点都有路径，则只需调用 1 次 DFS/BFS 函数
对于强连通图，从任一顶点出发都只需调用 1 次 DFS/BFS 函数

图遍历与连通性判断

对于非连通图，需要多次调用遍历函数。

> 调用 BFS/DFS 的次数 = 该图的**连通分量数**。

强连通图从任一顶点出发均可访问全图。有向图若非强连通，则可能需要多次调用。

最小生成树 (MST) 概念

对于带权连通无向图 G，权值之和最小的生成树称为 MST。

- 包含全部顶点，V-1 条边。
- **唯一性**：若图中各边权值均不等，则 MST 唯一。
- **权值唯一**：即使 MST 不唯一，权值之和也是固定的。

普里姆算法 (Prim)

逻辑： 从顶点开始增长。

1. 选取任一顶点加入集 U。
2. 找一条边 (u, v)，满足 $u \in U, v \notin U$，且权值最小。
3. 将 v 加入 U，直到 V=U。

复杂度：$O(|V|^2)$，适合存储方式：邻接矩阵。

适用场景

稠密图。时间复杂度与边数无关。

克鲁斯卡尔算法 (Kruskal)

逻辑： 从边开始增长。

1. 按权值从小到大排序所有边。
2. 依次选边，若不构成回路，则选入。
3. 利用**并查集**判断回路。

复杂度：$O(|E|\log|E|)$。

适用场景

稀疏图。主要时间花在边排序上。

最短路径问题分类

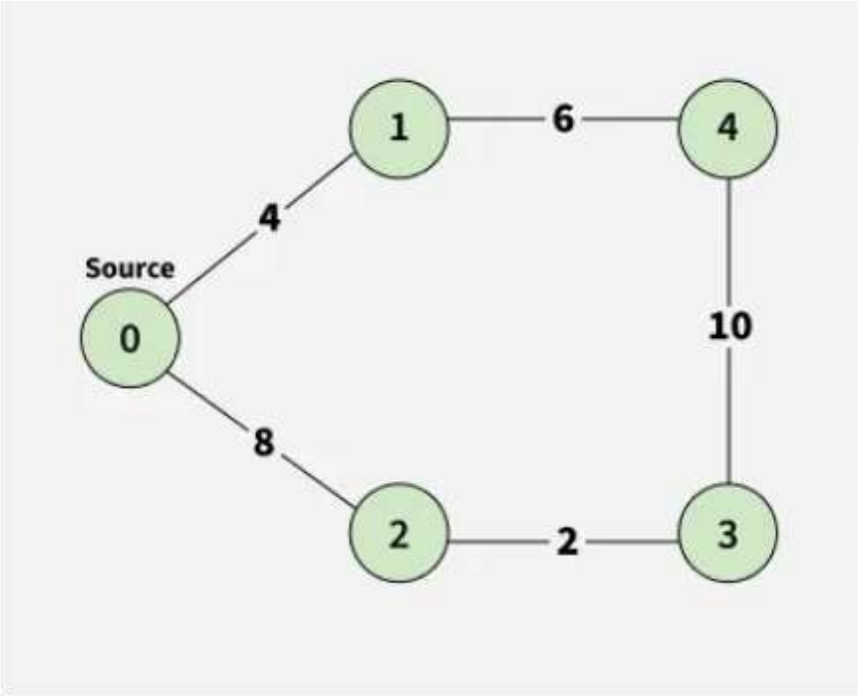
问题类型	典型算法	时间复杂度
单源最短路径 (无权图)	BFS	$O(V+E)$
单源最短路径 (带权图)	Dijkstra	$O(V^2)$
各对顶点间最短路径	Floyd	$O(V^3)$

狄克斯特拉算法 (Dijkstra)

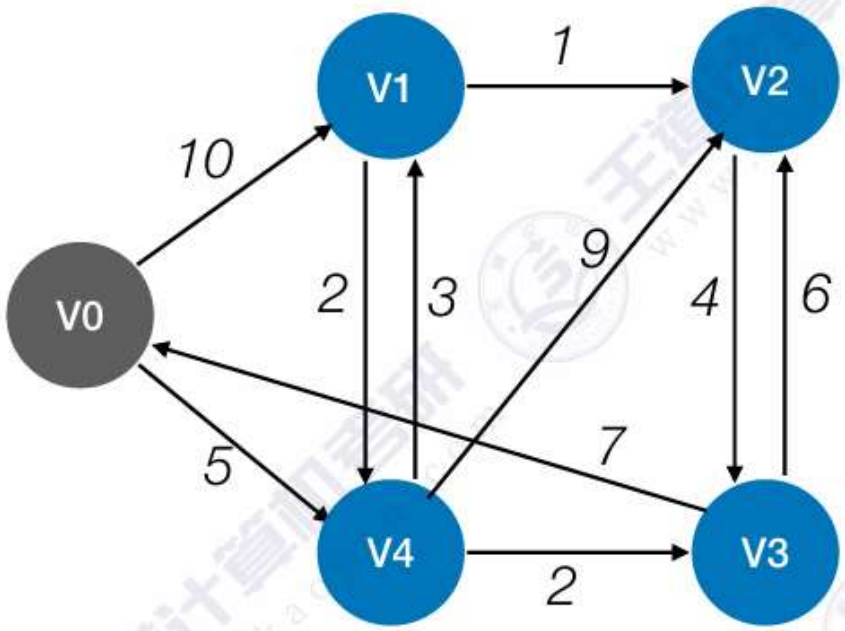
贪心策略。维护 dist[] 和 path[]。

🚨 **致命限制：** 不能处理带**负权边**的图。

复杂度：$O(V^2)$。若用优先队列优化可达 $O(E \log V)$。



Dijkstra算法的时间复杂度



标记各顶点是否已找到最短路径

	V0	V1	V2	V3	V4
final[5]	✓	✗	✗	✗	✗

最短路径长度

dist[5]	0	10	∞	∞	5

路径上的前驱

path[5]	-1	0	-1	-1	0

初始：若从V_0开始，令 final[0]=ture; dist[0]=0; path[0]=-1。
其余顶点final[k]=false; dist[k]=arcs[0][k]; path[k]= (arcs[0][k]==∞) ? -1 : 0

时间复杂度：$O(n^2)$即$O(|V|^2)$

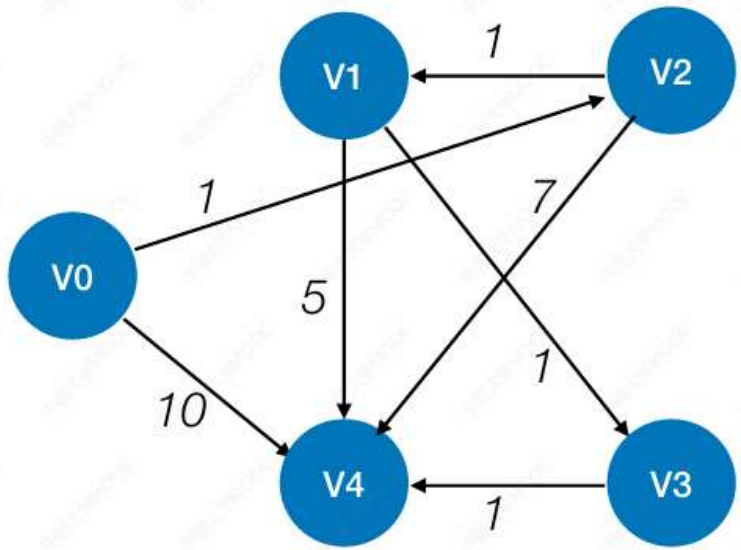
n-1轮处理：循环遍历所有顶点，找到还没确定最短路径，且dist 最小的顶点V_i，令final[i]=ture。并检查所有邻接自V_i 的顶点，对于邻接自V_i 的顶点 V_j，若 final[j]==false 且 dist[i]+arcs[i][j] < dist[j]，则令 dist[j]=dist[i]+arcs[i][j]; path[j]=i。（注：arcs[i][j]表示V_i 到V_j 的弧的权值）

弗洛伊德算法 (Floyd)

动态规划思想。通过引入中间节点逐步更新路径矩阵。

$$A^{(k)}[i][j] = \min\left(A^{(k-1)}[i][j], A^{(k-1)}[i][k] + A^{(k-1)}[k][j]\right)$$

- 可以处理负权边，但不能处理含**负权回路**的图。
- 核心优势：算法逻辑简单（三层嵌套循环）。



若　$A^{(k-1)}[i][j] > A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$

则　$A^{(k)}[i][j] = A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$;

　　$path^{(k)}[i][j] = k$

否则　$A^{(k)}$ 和 $path^{(k)}$ 保持原值

$A^{(3)}$ =

	V0	V1	V2	V3	V4
V0	0	2	1	3	4
V1	∞	0	∞	1	2
V2	∞	1	0	2	3
V3	∞	∞	∞	0	1
V4	∞	∞	∞	∞	0

$path^{(3)}$ =

	V0	V1	V2	V3	V4
V0	-1	2	-1	2	3
V1	-1	-1	-1	-1	3
V2	-1	-1	-1	1	3
V3	-1	-1	-1	-1	-1
V4	-1	-1	-1	-1	-1

#4：若允许在 V_0、V_1、V_2、V_3、V_4中转，最短路径是？——求 $A^{(4)}$ 和 $path^{(4)}$

```
for(int i=0; i<n; i++) {  //遍历整个矩阵，i为行号，j为列号
  for (int j=0; j<n; j++){
    if (A[i][j]>A[i][k]+A[k][j]){    //以 Vk 为中转点的路径更短
      A[i][j]=A[i][k]+A[k][j];       //更新最短路径长度
      path[i][j]=k;                  //中转点
    }
  }
}
```

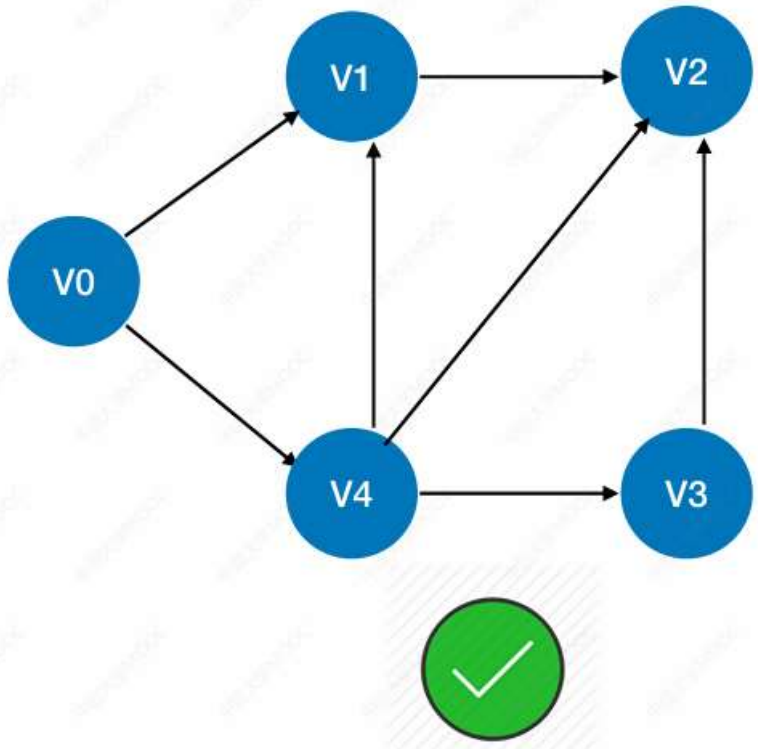
k=4

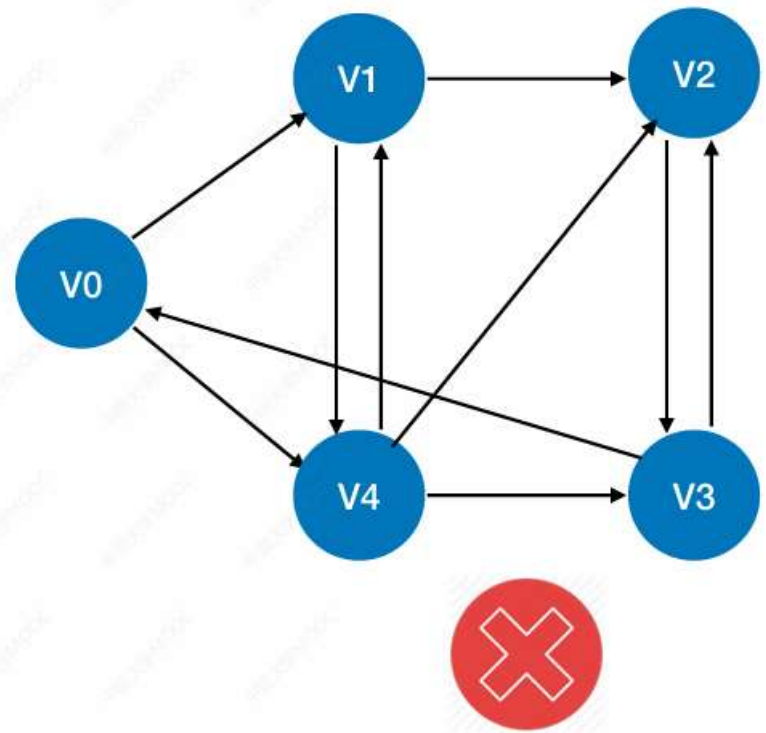
	BFS 算法	Dijkstra 算法	Floyd 算法
无权图	✓	✓	✓
带权图	✗	✓	✓
带负权值的图	✗	✗	✓
带负权回路的图	✗	✗	✗
时间复杂度	$O(\|V\|^2)$或$O(\|V\|+\|E\|)$	$O(\|V\|^2)$	$O(\|V\|^3)$
通常用于	求无权图的单源最短路径	求带权图的单源最短路径	求带权图中各顶点间的最短路径

注：也可用 Dijkstra 算法求所有顶点间的最短路径，重复 |V| 次即可，总的时间复杂度也是$O(|V|^3)$

有向无环图（DAG）

有向无环图：若一个有向图中不存在环，则称为有向无环图，简称DAG图（Directed Acyclic Graph）





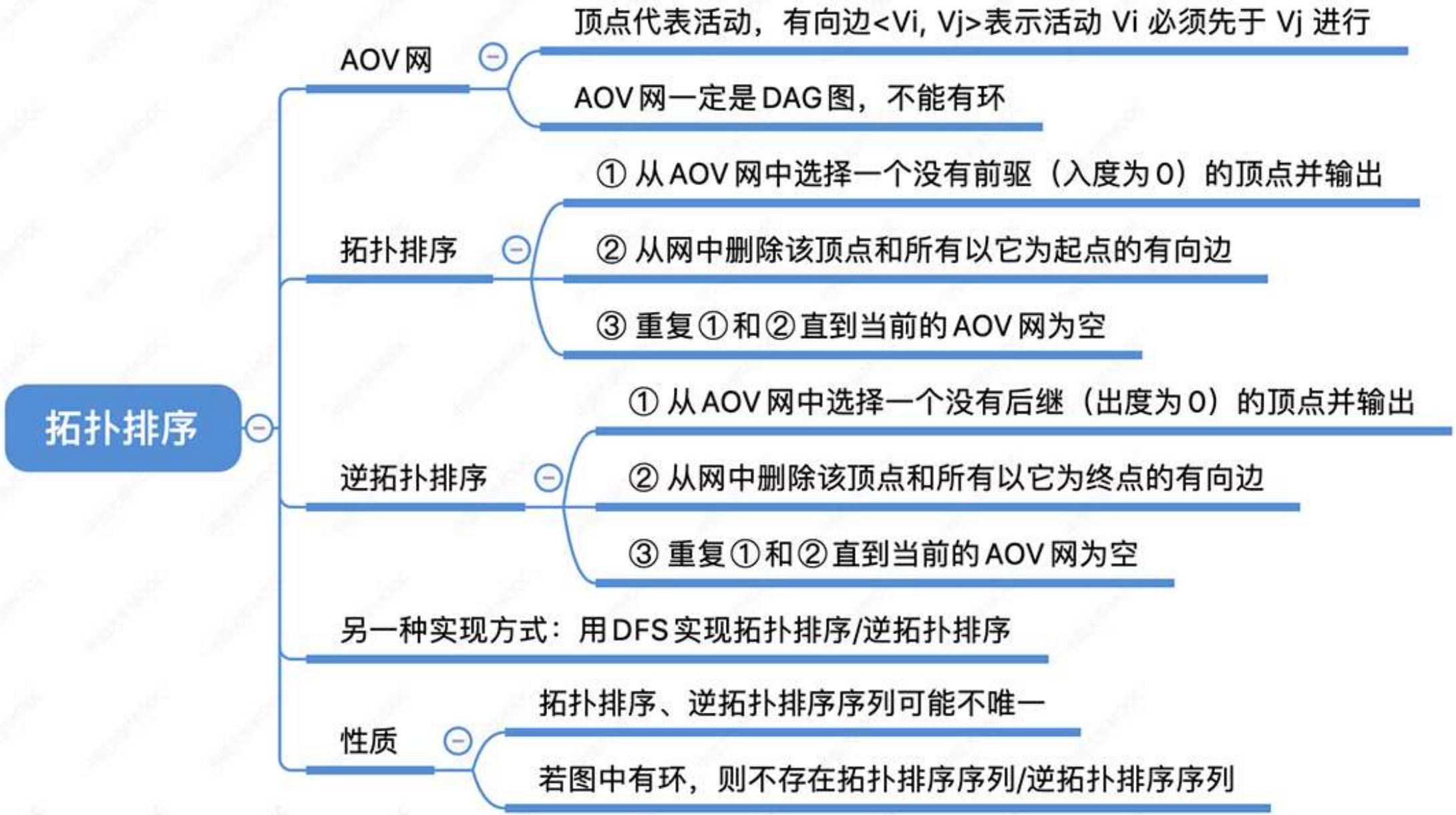
拓扑排序 (Topological Sort)

针对**有向无环图 (DAG)**。用于表示工程项目的先后顺序。

算法步骤

1. 选择一个入度为 0 的顶点输出。
2. 从图中删除该点及所有发出的弧。
3. 重复直到图空或不存在入度 0 顶点（说明有环）。

复杂度：O(V+E)。结果可能不唯一。


拓扑排序
AOV网
顶点代表活动，有向边<Vi, Vj>表示活动 Vi 必须先于 Vj 进行
AOV网一定是DAG图，不能有环
拓扑排序
① 从AOV网中选择一个没有前驱（入度为0）的顶点并输出
② 从网中删除该顶点和所有以它为起点的有向边
③ 重复①和②直到当前的AOV网为空
逆拓扑排序
① 从AOV网中选择一个没有后继（出度为0）的顶点并输出
② 从网中删除该顶点和所有以它为终点的有向边
③ 重复①和②直到当前的AOV网为空
另一种实现方式：用DFS实现拓扑排序/逆拓扑排序
性质
拓扑排序、逆拓扑排序序列可能不唯一
若图中有环，则不存在拓扑排序序列/逆拓扑排序序列

关键路径 (Critical Path) 概念

AOE 网：顶点表示事件，弧表示活动及持续时间。

- **关键路径**：从源点到汇点具有**最大长度**的路径。
- **关键活动**：关键路径上的活动，时间余量为 0。

> 🚨 **要点**：关键路径长度 = 完成工程的最短时间。

关键路径计算的四个量

ve(i)	事件最早发生：源点到该点的最长路径。
vl(i)	事件最迟发生：汇点 ve 减该点到汇点最长路径。
e(k)	活动最早开始：对应弧尾结点的 ve。
l(k)	活动最迟开始：对应弧头结点的 vl 减弧权值。

① 求所有事件的最早发生时间 ve( )
② 求所有事件的最迟发生时间 vl( )
③ 求所有活动的最早发生时间 e( )
④ 求所有活动的最迟发生时间 l( )
⑤ 求所有活动的时间余量 d( )

*d(i)=0*的活动就是关键活动，由关键活动可得关键路径

①按拓扑排序序列，依次求各个顶点的 ***ve(k)*:**

ve(源点) = 0

$ve(k) = \text{Max}\{ve(j) + \text{Weight}(v_j, v_k)\}$，　v_j为v_k的任意前驱

②按逆拓扑排序序列，依次求各个顶点的 ***vl(k)*:**

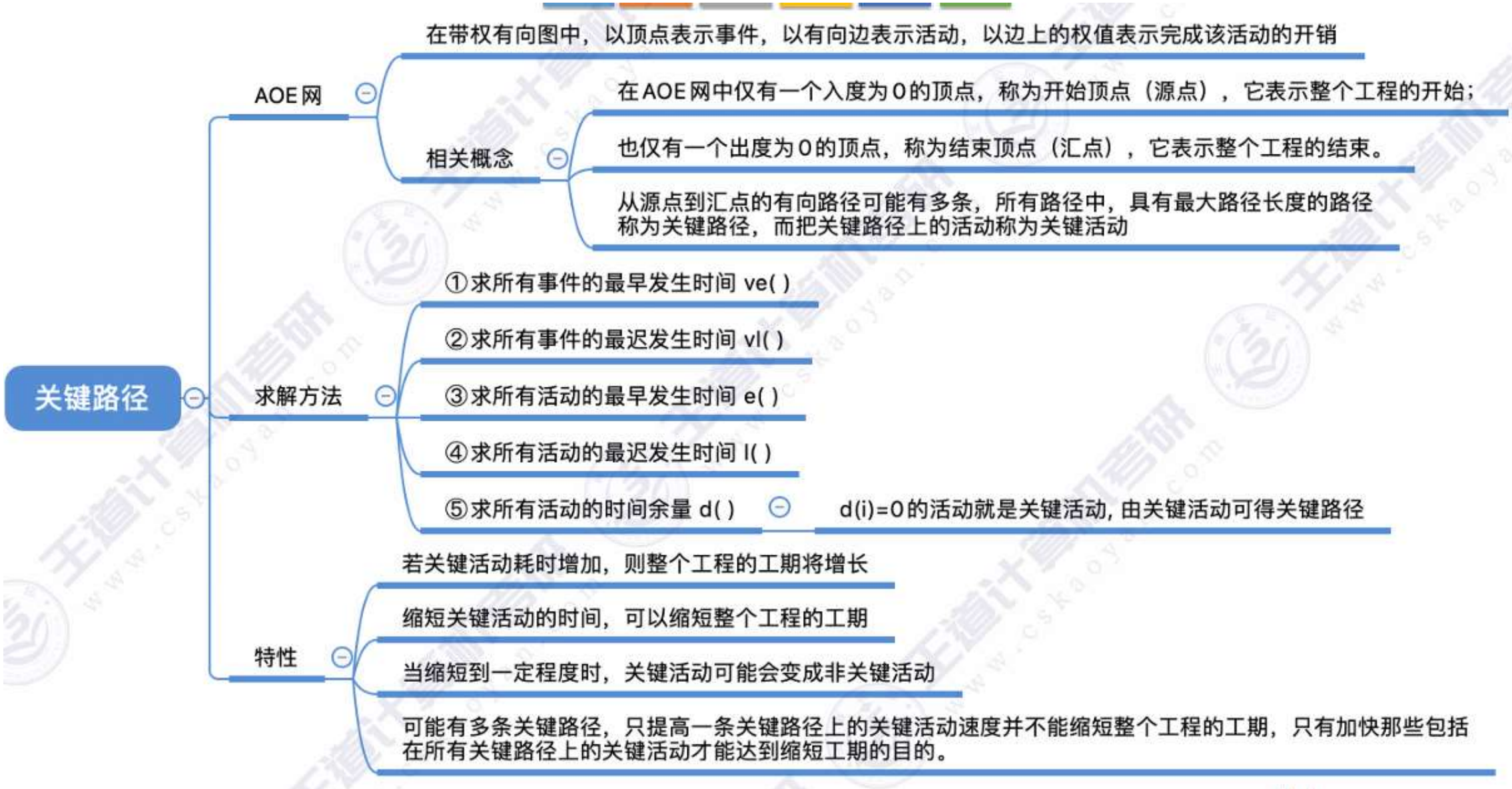
vl(汇点) = ve(汇点)

$vl(k) = \text{Min}\{vl(j) - \text{Weight}(v_k, v_j)\}$, v_j为v_k的任意后继

③若边<v_k, v_j>表示活动a_i，则有$e(i) = ve(k)$

④若边<v_k, v_j>表示活动a_i，则有$l(i) = vl(j) - \text{Weight}(v_k, v_j)$

⑤$d(i) = l(i) - e(i)$


关键路径
AOE网
在带权有向图中，以顶点表示事件，以有向边表示活动，以边上的权值表示完成该活动的开销
相关概念
在AOE网中仅有一个入度为0的顶点，称为开始顶点（源点），它表示整个工程的开始；
也仅有一个出度为0的顶点，称为结束顶点（汇点），它表示整个工程的结束。
从源点到汇点的有向路径可能有多条，所有路径中，具有最大路径长度的路径
称为关键路径，而把关键路径上的活动称为关键活动
求解方法
①求所有事件的最早发生时间 ve( )
②求所有事件的最迟发生时间 vl( )
③求所有活动的最早发生时间 e( )
④求所有活动的最迟发生时间 l( )
⑤求所有活动的时间余量 d( )
d(i)=0的活动就是关键活动，由关键活动可得关键路径
特性
若关键活动耗时增加，则整个工程的工期将增长
缩短关键活动的时间，可以缩短整个工程的工期
当缩短到一定程度时，关键活动可能会变成非关键活动
可能有多条关键路径，只提高一条关键路径上的关键活动速度并不能缩短整个工程的工期，只有加快那些包括
在所有关键路径上的关键活动才能达到缩短工期的目的。

18.【2017 统考真题】已知无向图 G 含有 16 条边，其中度为 4 的顶点个数为 3，度为 3 的顶点个数为 4，其他顶点的度均小于 3。图 G 所含的顶点个数至少是（ ）。

A. 10　　B. 11　　C. 13　　D. 15

18. B

无向图边数的 2 倍等于各顶点度数的总和。要求至少的顶点数，应使每个顶点的度取最大，而由于其他顶点的度均小于 3，因此可设它们的度都为 2，并设它们的数量是 x，列出方程 $4\times3+3\times4+2x=16\times2$，解得 $x=4$。因此至少包含 $4+4+3=11$ 个顶点。

18. 【2013统考真题】设图的邻接矩阵 A 如下所示，各顶点的度依次是（ ）。

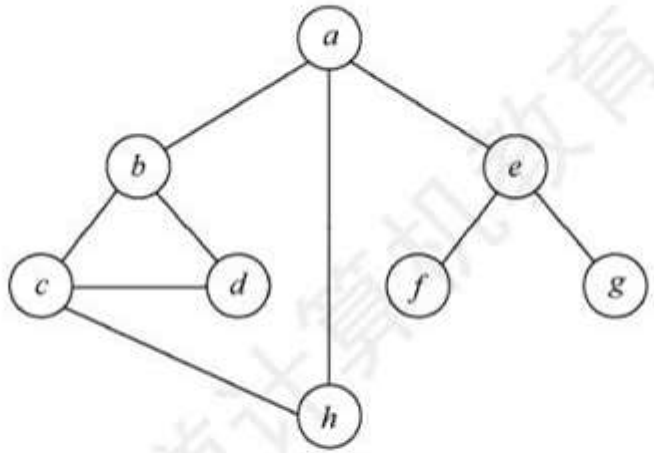
$$A=\begin{bmatrix}0&1&0&1\\0&0&1&1\\0&1&0&0\\1&0&0&0\end{bmatrix}$$

A. 1, 2, 1, 2　　B. 2, 2, 1, 1　　C. 3, 4, 2, 3　　D. 4, 4, 2, 2

18. C

邻接矩阵 A 为非对称矩阵，说明图是有向图，度为入度与出度之和。各顶点的度是矩阵中此结点对应的行（对应出度）和列（对应入度）的非零元素之和。

16. 【2013 统考真题】下列选项中，不是如下无向图的广度优先遍历序列的是（ ）。



A. h, c, a, b, d, e, g, f

B. e, a, f, g, b, h, c, d

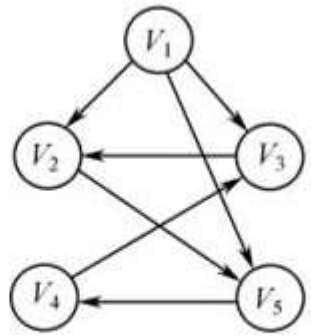
C. d, b, c, a, h, e, f, g

D. a, b, c, d, h, e, f, g

16. D

只要掌握 DFS 和 BFS 的遍历过程，便能轻易解决。逐个代入，手工模拟，选项 D 是深度优先遍历，而不是广度优先遍历。

18. 【2016 统考真题】下列选项中，不是下图深度优先搜索序列的是（ ）。



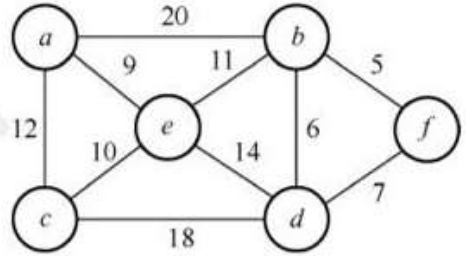
A. V_1, V_5, V_4, V_3, V_2　　B. V_1, V_3, V_2, V_5, V_4

C. V_1, V_2, V_5, V_4, V_3　　D. V_1, V_2, V_3, V_4, V_5

18. D

按深度优先遍历的策略进行遍历。对于 A：先访问 V_1，然后访问与 V_1 邻接且未被访问的任意一个顶点（满足的有 V_2, V_3 和 V_5），此时访问 V_5，然后从 V_5 出发，访问与 V_5 邻接且未被访问的任意一个顶点（满足的只有 V_4），然后从 V_4 出发，访问与 V_4 邻接且未被访问的任意一个顶点（满足的只有 V_3），然后从 V_3 出发，访问与 V_3 邻接且未被访问的任意一个顶点（满足的只有 V_2），结束遍历。B 和 C 的分析方法与 A 相同。对于 D，首先访问 V_1，然后从 V_1 出发，访问与 V_1 邻接且未被访问的任意一个顶点（满足的有 V_2, V_3 和 V_5），然后从 V_2 出发，访问与 V_2 邻接且未被访问的任意一个顶点（满足的只有 V_5），按规则本应该访问 V_5，但 D 却访问了 V_3，错误。

40.【2020 统考真题】已知无向图 G 如下所示，使用 Kruskal 算法求图 G 的最小生成树，加到最小生成树中的边依次是（ ）。

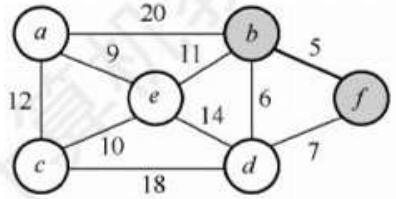


A. $(b,f),(b,d),(a,e),(c,e),(b,e)$　　B. $(b,f$

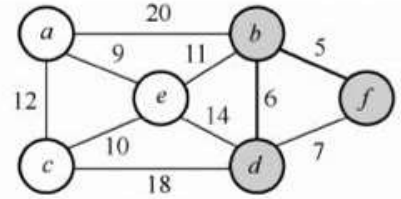
C. $(a,e),(b,e),(c,e),(b,d),(b,f)$　　D. $(a,e$

40. A

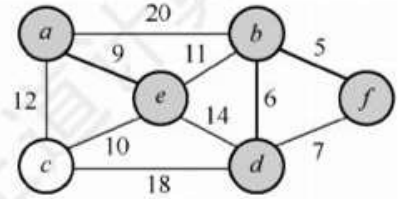
Kruskal 算法：按权值递增顺序依次选取 $n-1$ 条边，并保证这 $n-1$ 条边不构成回路。初始构造一个仅含 n 个顶点的森林；第一步，选取权值最小的边(b,f)加入最小生成树；第二步，剩余边中权值最小的边为(b,d)，加入最小生成树，第二步操作后权值最小的边(d,f)不能选，因为会与之前已选取的边形成回路；接下来依次选取权值 9、10、11 对应的边加入最小生成树，此时 6 个顶点形成了一棵树，最小生成树构造完成。按照上述过程，加到最小生成树的边依次为(b,f)，$(b,d),(a,e),(c,e),(b,e)$。其生成过程如下所示。



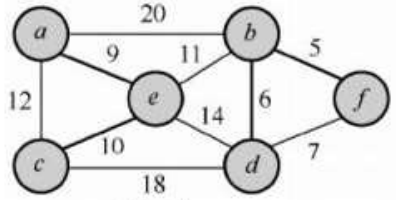
第一步
选取边<b,f>



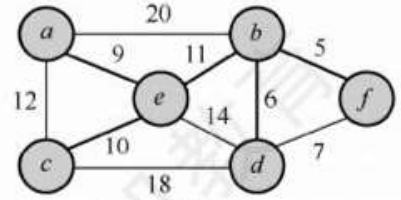
第二步
选取边<b,d>



第三步
选取边<a,e>

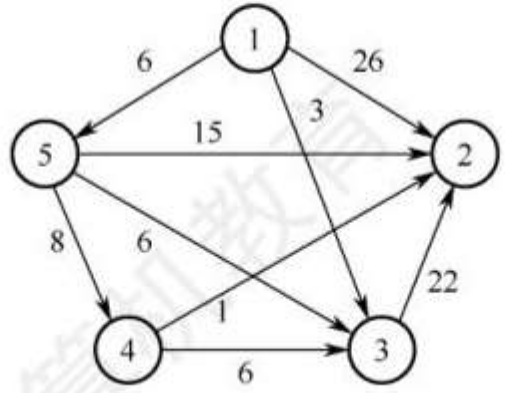


第四步
选取边<c,e>



第五步
选取边<b,e>

44. 【2021 统考真题】使用 Dijkstra 算法求下图中从顶点 1 到其余各顶点的最短路径，将当前找到的从顶点 1 到顶点 2, 3, 4, 5 的最短路径长度保存在数组 dist 中，求出第二条最短路径后，dist 中的内容更新为（ ）。

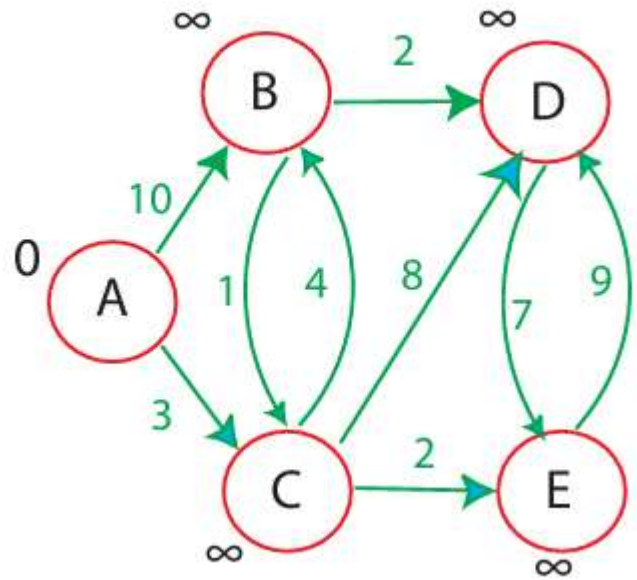


A. 26, 3, 14, 6　　B. 25, 3, 14, 6　　C. 21, 3, 14, 6　　D. 15, 3, 14, 6

44. C

在执行 Dijkstra 算法时，首先初始化 dist[]，若顶点 1 到顶点 i（$i = 2, 3, 4, 5$）有边，就初始化为边的权值；若无边，就初始化为∞；初始化顶点集 S 只含顶点 1。Dijkstra 算法每次选择一个到顶点 1 距离最近的顶点 j 加入顶点集 S，并判断由顶点 1 绕行顶点 j 后到任意一个顶点 k 是否距离更短，若距离更短（即 dist[j] + arcs[j][k] < dist[k]），则将 dist[x]更新为 dist[j] + arcs[j][k]；重复该过程，直至所有顶点都加入顶点集 S。数组 dist 的变化过程如下图所示，可知将第二个顶点 5 加入顶点集 S 后，数组 dist 更新为 21, 3, 14, 6。

$$\text{dist}\{26, 3, \infty, 6\} \xrightarrow{\text{顶点3入S}} \{25, 3, \infty, 6\} \xrightarrow{\text{顶点5入S}} \{21, 3, 14, 6\}$$



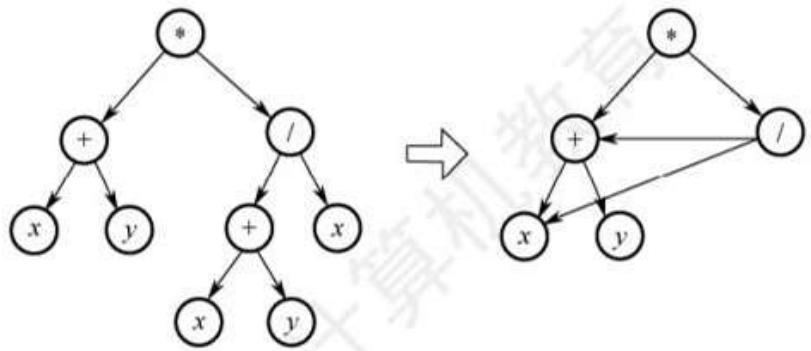
S = { }	{ A B C D E } =	Q	
S = { A }	0 ∞ ∞ ∞ ∞		
S = { A, C }	0 10 3 ∞ ∞	←	after relaxing edges from A
S = { A, C }	0 7 3 11 5	←	after relaxing edges from C
S = { A, C, E }	0 7 3 11 5		
S = { A, C , E, B}	0 7 3 9 5	←	after relaxing edges from B

39. 【2019 统考真题】用有向无环图描述表达式 $(x+y)((x+y)/x)$，需要的顶点个数至少是（ ）。

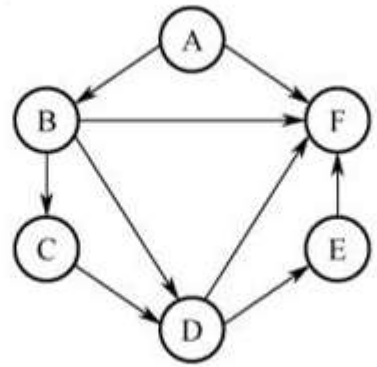
A. 5　　B. 6　　C. 8　　D. 9

39. A

先将该表达式转换成有向二叉树，该二叉树中有些顶点是重复的，为了节省存储空间，去除重复的顶点，将有向二叉树去重转换成有向无环图，如下图所示。



43.【2021统考真题】给定如下有向图，该图的拓扑有序序列的个数是（ ）。

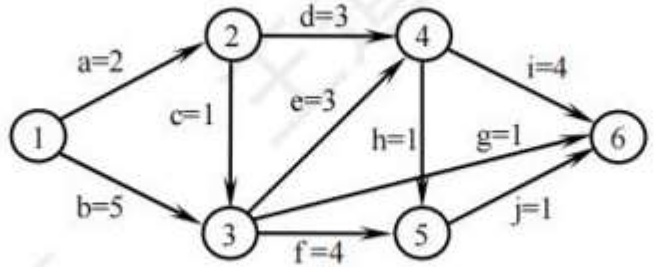


A. 1　　B. 2　　C. 3　　D. 4

43. A

求拓扑序列的过程：从图中选择无入边的结点，输出该结点并删除该结点的所有出边，重复上述过程，直至全部结点都已输出，这样求得的拓扑序列为 ABCDEF。每次输出一个结点并删除该结点的所有出边后，都发现有且仅有一个结点无入边，因此该拓扑序列唯一。

45. 【2022 统考真题】下图是一个有 10 个活动的 AOE 网，时间余量最大的活动是（ ）。



A. c　　B. g　　C. h　　D. j

45. B

在 AOE 网中，活动的时间余量＝结束顶点的最迟开始时间－开始顶点的最早开始时间－该活动的持续时间。根据关键路径算法得到下表：

结点编号	1	2	3	4	5	6
最早开始时间 $v_e(i)$	0	2	5	8	9	12
最迟开始时间 $v_l(i)$	0	4	5	8	11	12

c 的时间余量＝$v_l(3)-v_e(2)-1=5-2-1=2$，g 的时间余量＝$v_l(6)-v_e(3)-1=12-5-1=6$，h 的时间余量＝$v_l(5)-v_e(4)-1=11-8-1=2$，j 的时间余量＝$v_l(6)-v_e(5)-1=12-9-1=2$。